

# PKU-PvZ 项目报告

## 1. 程序功能介绍

本项目是一个基于 C++ 与 Qt6 开发的《植物大战僵尸》魔改版小游戏，项目采用 CMake 构建，工程目录主要分为 core、ui、animation、resources 等模块，分别负责游戏核心逻辑、界面交互、动画管理和资源组织。

游戏主要提供两种玩法模式。第一种是经典植物防守模式：玩家通过卡牌选择植物，在草坪格子中种植植物，消耗阳光抵御从右侧出现的僵尸。可选植物包括“不知火蛙 Firefan”“金日葵 Kimsunflower”“熊绷果 Bengbear”“带派辣椒 Rainchili”等。植物放置需要满足格子为空、阳光足够、冷却结束等条件；游戏中会自动生成阳光，向日葵也会周期性生产阳光，玩家收集阳光后可以继续种植。

第二种是僵尸模式：玩家不再扮演植物方，而是选择僵尸卡牌，在右侧可放置区域派出僵尸，目标是清空场上植物。该模式启动时，程序会在草坪左侧自动生成若干植物阵容；玩家只能在第 5 列及其右侧放置僵尸。可选僵尸包括“艮子僵尸 GenziZombie”和“舞王僵尸 DancingZombie”。

游戏包括主菜单、游戏说明、游戏界面、暂停、铲子铲除、游戏结束、重新开始和返回主菜单等功能。MainWindow 使用 QStackedWidget 在主菜单、说明页、游戏页、结束页之间切换，并在开始游戏时创建 GameEngine、GamePageWidget 和 AnimationManager。游戏还接入了背景音乐，通过 QMediaPlayer 循环播放资源中的 bgm.mp3。

## 2. 项目各模块与类设计细节

### 2.1 总体架构设计

项目整体采用“核心逻辑 + 图形界面 + 动画资源”的分层设计。core 模块维护游戏实体、地图格子、阳光、子弹、植物、僵尸和主循环；ui 模块负责窗口、卡牌、顶部状态栏、游戏画布、结束页和说明页；animation 模块负责帧动画、阳光收集动画、死亡动画、子弹命中特效等；resources 模块存放图片、音乐等素材。仓库中 core 目录包含 GameEngine、GameEntity、GridManager、Plant、Zombie、Bullet、Sun 等文件，ui 目录包含 GameView、CardPanel、PlantCardWidget、TopBarWidget、MainMenuWidget、GuideWidget、GameOverWidget 等文件，animation 目录包含 AnimationManager、SpriteAnimation、ResourceManager、EffectItem 等文件。

模块之间主要通过 Qt 的信号槽机制解耦。GameEngine 负责发出实体创

建、实体死亡、阳光变化、波次变化、动画状态变化、游戏结束等信号；GameView 接收实体并加入场景，AnimationManager 接收动画请求，TopBarWidget 和 CardPanel 更新阳光数值与卡牌状态。这样可以避免游戏逻辑直接控制界面绘制，使核心规则与表现层相对独立。

## 2.2 核心逻辑模块 core

GameEntity 是所有游戏对象的基类，继承自 QObject 和 QGraphicsPixmapItem，统一保存行列坐标、生命值、最大生命值、存活状态、死亡动画完成状态以及实体类型。它提供 updateEntity()、takeDamage()、die()、isAlive() 等通用接口，植物、僵尸、子弹和阳光都可以基于此类实现统一的生命管理和场景显示。

Plant 是植物基类，保存植物类型和种植花费，并提供攻击、生产阳光等虚函数。具体植物类通过继承实现差异化能力：Firefan 具备攻击计时器和创建子弹能力；Bengbear 主要承担高生命值防御；Kimsunflower 通过生产计时器周期性产生阳光；Rainchili 种下后进入蓄力状态，达到引爆时间后消灭同一行僵尸并自毁。

Zombie 是僵尸基类，包含移动速度、攻击伤害、攻击间隔、攻击状态和目标植物指针。普通僵尸 GenziZombie 负责基础推进；DancingZombie 额外持有召唤计时器，能周期性进入召唤状态，并在召唤完成后生成伴舞僵尸 DancerZombie。这让僵尸侧不只是简单移动，还具有战术扩张能力。

GridManager 负责维护 5 行 9 列的草坪格子，提供格子合法性判断、格子是否为空、放置植物、移除植物、获取格子植物，以及场景坐标和格子坐标之间的转换。它是“鼠标点击位置”和“游戏规则格子”之间的桥梁，也是防止同一格重复种植的关键类。

GameEngine 是核心控制器，内部维护植物列表、僵尸列表、子弹列表、阳光列表、格子管理器、主循环计时器、阳光生成计时器、僵尸生成计时器、当前阳光值、当前波次、总波次、植物冷却表等状态。它提供开始、暂停、恢复、停止、放置植物、放置僵尸、铲除植物、更新游戏、碰撞检测、死亡清理、波次切换和胜负判定等接口。

游戏主循环通过 updateGame() 周期执行，依次处理植物攻击、阳光生产、辣椒引爆、僵尸攻击植物、子弹碰撞、僵尸移动、子弹更新、死亡对象清理、冷却刷新、波次切换和胜负判定。该循环体现了实时游戏中“状态更新—碰撞判定—资源回收—界面通知”的基本流程。

胜负逻辑根据模式区分：经典模式下，如果有僵尸到达最左侧，则玩家失败；当当前波次僵尸全部生成并被消灭后进入下一波，全部波次结束后玩家胜利。僵尸模式下，如果场上植物全部死亡，则僵尸方获胜。

### 2.3 界面交互模块 ui

MainWindow 是应用的总窗口，负责主菜单、说明页、游戏页、结束页之间的切换，并统一创建和销毁游戏对象。它在开始游戏时将 GameEngine、GameView、CardPanel、TopBarWidget、AnimationManager 连接起来，是整个程序的页面调度和信号槽装配中心。

GamePageWidget 是游戏界面的容器类，由顶部状态栏 TopBarWidget、卡牌面板 CardPanel 和游戏画布 GameView 三部分组成。该设计把操作界面、状态显示和主场景区域分离，使界面布局较为清晰。

GameView 继承自 QGraphicsView，负责初始化 QGraphicsScene、绘制背景、绘制草坪格子、添加和移除实体图元、将鼠标点击转换为格子坐标，并根据当前模式发出植物放置、僵尸放置或铲除植物的信号。在僵尸模式下，GameView 会高亮右侧可投放区域，并限制僵尸只能在右侧列放置。

CardPanel 和 PlantCardWidget 负责卡牌选择与状态显示。CardPanel 可以添加植物卡和僵尸卡，记录当前选中的卡牌类型，并根据阳光值和冷却时间刷新卡牌状态；PlantCardWidget 则负责单张卡牌的绘制、点击、是否可购买、是否选中和冷却显示。

### 2.4 动画与资源模块 animation/resources

AnimationManager 统一管理实体动画状态，定义了 Idle、Walk、Attack、Hit、Death、Produce、Spawn、Collect、Move 等动画状态，并使用不同 Z 层级控制背景、植物、僵尸、子弹、阳光、特效和 UI 的显示顺序。它根据实体类型绑定对应动画，并提供阳光生成、阳光收集、死亡特效、子弹命中特效等动画接口。

SpriteAnimation 封装了帧动画的播放逻辑，保存目标图元、帧序列、计时器、当前帧、帧间隔、循环标志和运行状态，并提供开始、停止、重置和下一帧切换等接口。ResourceManager 则负责加载单张图片、批量加载动画帧，以及在资源缺失时生成占位图，降低资源加载失败对游戏运行的影响。

## 3. 小组成员分工情况

| 成员  | 负责内容        | 具体工作                                           |
|-----|-------------|------------------------------------------------|
| 张兆祺 | 核心逻辑与植物僵尸系统 | 设计 GameEntity、GameEngine 及相关游戏机制，实现植物和僵尸基类与派生类 |

| 成员  | 负责内容        | 具体工作                                                        |
|-----|-------------|-------------------------------------------------------------|
| 刘子泰 | 设计图形界面与交互功能 | 实现主菜单、说明页、游戏页、卡牌面板、顶部状态栏、鼠标点击放置、铲子模式、暂停和结束界面等               |
| 湛浩文 | 完成动画功能并整理资源 | 实现 AnimationManager、SpriteAnimation 等，整理图片与音乐资源，完成动画绑定与背景音乐 |

#### 4. 项目总结与反思

本项目较好地完成了从命令式逻辑到图形化小游戏的转化。代码中使用继承和多态描述植物、僵尸、子弹、阳光等实体，用 GameEngine 统一调度核心规则，用 Qt 的信号槽机制连接逻辑层、界面层和动画层，整体结构清晰，便于后续增加新植物、新僵尸或新关卡。经典模式和僵尸模式的加入，也让项目不只是复刻原版玩法，而是具有一定的创新性。

项目的亮点主要体现在三个方面。第一，核心规则较完整，包含阳光、冷却、攻击、碰撞、波次、胜负和暂停恢复。第二，界面模块拆分清楚，主窗口、游戏画布、卡牌、顶部栏和结束界面各司其职。第三，动画系统相对独立，实体只负责状态变化，动画播放由 AnimationManager 统一处理，降低了表现层和逻辑层的耦合。

项目也存在一些可以改进的地方。首先，代码中较多使用裸指针和手动 delete，虽然当前通过死亡清理和 deleteLater() 管理对象生命周期，但随着实体数量和玩法复杂度提升，可能出现悬空指针或重复释放风险，后续可以考虑引入智能指针、Qt 父子对象管理或更明确的对象所有权规范。其次，部分字符串类型如 "Firefan"、"Kimsunflower"、"GenziZombie" 等直接散落在代码中，后续可改为枚举或配置表，提高可维护性。再次，部分坐标转换中行列顺序在不同类中表达方式不完全一致，虽然当前逻辑能够运行，但后续维护时容易误用，应统一接口命名和返回语义。最后，游戏平衡仍有提升空间，例如僵尸刷新闻隔、阳光产量、植物血量与冷却时间可以通过配置文件或关卡数据表调整，而不是固定写在代码中。

总体来看，本项目完成了一个较完整的 Qt 图形化塔防游戏框架，覆盖了面向对象设计、事件驱动编程、图形场景管理、动画管理、资源加载和团队协作等多个课程重点。通过本项目，小组成员对 C++ 类设计、Qt 信号槽、实时游戏循环以及模块化开发有了更深入的理解。后续如果继续完善，可以增加关卡选择、更多植物僵尸类型、存档系统、音效管理、难度曲线、单元测试和更规范的资源配置，使项目从课程大作业进一步发展为更成熟的小型游戏作品。